

Is the browser the bottleneck? A capability-ceiling study of policy-conditioned, fully-local detection of confidential data in LLM prompts

AI-DLPP research prototype · draft v1 · 8 September 2026 · *author and affiliation to be added* · every figure and table in this paper regenerates from the run artifacts with docs/paper/extract-numbers.py and make-figures.py

ABSTRACT. AI-DLPP is a browser extension that detects confidential data in prompts to LLM services entirely on the user's machine, under a natural-language leak policy compiled once into a versioned intermediate representation, and pseudonymises, redacts or blocks before anything leaves the browser. Its shippable detectors — 2–4 B-parameter models on WebGPU — find the right region of text and attach the wrong label. This paper asks whether that is the browser's fault or the task's. We run the same compiled judge and the same policy-in-context baseline, unchanged, against six open-weight hosted models with providers pinned, at temperature 0, in three passes, with a trivial floor beside every figure and every number re-derived from a reconciled artifact. Scored the way the study was designed — span-matched — the hosted arms land on the floor (0.602 vs 0.571). Scored on the message-level

judgment the gold actually records, they clear it in every pass (mean 0.881 vs 0.776) while the in-browser arms stay far below it (0.262): on that reading the gap is substantially the price of the in-browser constraint. Pass-to-pass variance at temperature 0 is 6× larger for classification than for span extraction. Giving one mandatory-reasoning model room to finish (8,192 tokens instead of 600) takes its recall from 0.579 to 1.000 and makes it the only arm to beat a label-informed span oracle. Provider-side constrained decoding produced zero malformed responses in 5,580 calls; the compiled judge is 2.05× cheaper and 2.1–3.7× faster than policy-in-context on the same model; the whole study cost \$1.22. Six defects in our own instrumentation are reported, including the one that reversed the original conclusion.

1 Introduction

Employees paste confidential material into consumer LLM interfaces. AI-DLPP intercepts each prompt (and text-file upload) to ChatGPT, Claude, Gemini or DeepSeek, detects confidential data *locally*, and applies a policy-driven action — allow, pseudonymise, redact or block — with pseudonymised values transparently restored in the streamed reply. Two properties are fixed by design. **Policy-adaptivity:** the taxonomy of "confidential" is not baked into weights; a natural-language policy document, including provider-specific clauses ("customer data may not go to foreign-hosted services"), shapes the whole behaviour at runtime. **The cloud boundary:** user prompts never leave the browser; a frontier model is permitted only at policy-compile time, which touches no user data.

The project's research question is *how well can policy-conditioned, fully-local models prevent confidential-data leakage in LLM prompts, and at what latency and hardware cost?* The first bake-off of in-browser models answered the first half badly: no shippable arm beat a trivial capitalisation rule at the policy's semantic predicate, and a two-rule orthographic reader was competitive with every arm at span finding. That left the design's central question unanswerable with the arms available (spec §4.2b): **is the browser the bottle-**

neck, or is small-model capability the bottleneck? If a much larger model run by the identical method also performs poorly, the task is hard and the browser constraint is not what costs accuracy; if it performs well, the gap between it and the best browser arm is the price of the in-browser constraint — the accuracy-versus-hardware trade the project set out to quantify.

This paper reports that capability-ceiling study. Its contributions are: (i) the ceiling arm itself — six open-weight models on pinned providers, five of them through three thinking-off passes at temperature 0 (5,580 calls) and one thinking-ON, reconciled to the cent; (ii) the finding that the answer to §4.2b depends on a metric-granularity choice the original design never labelled as a choice, and that the gold's own unit says the browser *is* the bottleneck; (iii) a variance result — classification is 6× less stable than span extraction across identical passes — that independently confirms *where* these models fail; (iv) a thinking-ON result showing the ceiling measured with reasoning disabled is a ceiling on that inference mode, not on the models; (v) latency, cost and structured-output measurements that transfer to a local-server deployment; and (vi) six measurement defects found in our own instrumentation, stated rather than fixed silently.

2 The system under test

Compile once in the cloud; detect every prompt locally

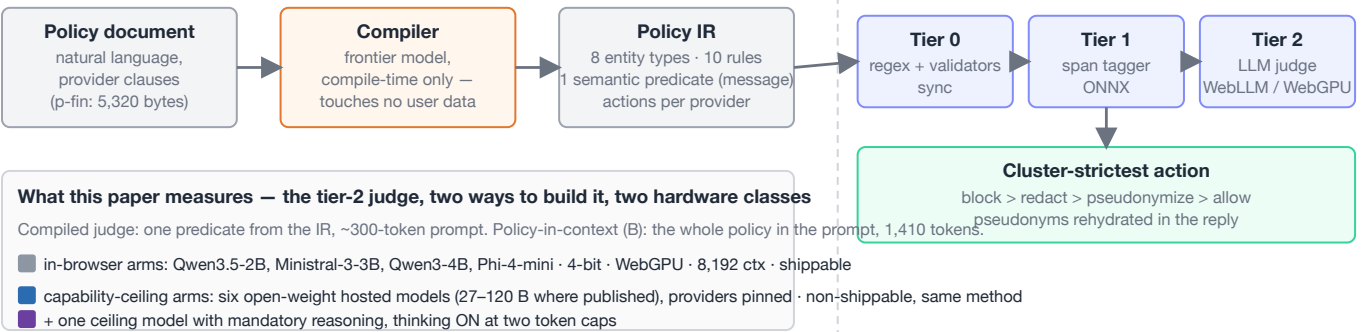


Figure 1. The pipeline, and the two things this paper varies. A policy is compiled once (cloud permitted, no user data) into an IR; every prompt is then checked locally by three tiers and actioned by the strictest rule in its cluster. The tier-2 judge is built two ways — a compiled, single-predicate judge and a policy-in-context baseline (Approach B) — and run on two hardware classes: shippable in-browser models and non-shippable hosted models that use the identical method and transport nothing else.

2.1 Policy → IR

The compiler turns a policy document into a hash-stamped JSON IR: entity types with tier, severity and surrogate kind; rules; semantic predicates in natural language with a declared scope; per-provider action overrides; fail mode; and a latency budget. The policy studied here, p-fin (5,320 bytes), compiles to eight entity types, ten rules and one message-scoped predicate, *client-relationship-disclo-*

sure (carried in the IR as a ninth, shadow entity type), whose action is *redact* and which asks whether a message discloses that the firm acts for, is pitching, or is counterparty to a named organisation. The policy-adaptivity metric proper (three deliberately disagreeing policies) is out of scope here: p-med and p-corp have no compiled IR yet.

2.2 Tiered runtime

Tier 0 is compiled regex plus validators and entropy checks, synchronous. Tier 1 is a span tagger on ONNX. Tier 2 is an instruct model that answers the semantic predicates, run in-browser via WebLLM on WebGPU with grammar-constrained JSON output. A finding names a quoted clause and a shorter mention inside it; a two-rung span-recovery ladder (exact folded match, then code-point peel) locates the action span. Predicates *partition* by declared scope, so a message-scoped policy costs exactly one judge call per message. Findings are clustered and the strictest action wins (block > redact > pseudonymise > allow).

2.3 Two ways to build the judge

The **compiled judge** receives one predicate from the IR and the passage — 252–280 prompt tokens on the smoke corpus, 408–447 on the study corpus. **Approach B** receives the entire policy document and is asked about all nine classes at once — 1,410 tokens. Both use the same JSON schema, the same span ladder and the same action resolution; B is additionally handed the compiler's entity-type ids, without which none of its findings would normalise, so it is not a pure "no compiler" arm.

3 Experimental design

3.1 Corpus and gold

The injection corpus (v2) has 189 messages carrying 108 entity gold spans over eight types (API credentials, bank identifiers, client names, DB connection strings, Indian PAN and Aadhaar numbers, internal customer ids, private-key material); 57.1% of messages carry at least one. Gold spans are arithmetic over the splice, never located by search, and generation refuses unless the slice reproduces the value. The **predicate gold** comes from a blind round: two annotators in isolated directories, forbidden the compiled IR, answered one question per message — *does it satisfy the predicate* — for all 189 rows, with a supplementary span. They agreed on every row; ten rows were nonetheless excluded as *disputed with the document*, because the policy's operative word "counterparty" does not settle whether an organisation that sells *to* the firm is caught (both annotators marked all ten borderline). The scored gold is therefore **179 rows with 19 positives and 19 spans**. Two honest limits: agreement is agreement within a shared framing, and the annotator brief unintentionally paraphrased the predicate in the IR's own wording; and the corpus's known generator leak means every positive contains a capitalised organisation name — all 19 gold spans match [A–Z] [a–z]+([A–Z] [a–z]+)+ exactly — so a capitalisation reader cannot miss a positive here and no floor figure transfers to a corpus without that property.

3.2 Arms

In-browser (shippable). Four 4-bit models on WebGPU in real Chrome via Playwright — Qwen3.5-2B, Ministral-3-3B, Qwen3-4B, Phi-4-mini (2.2–3.4 GB VRAM) — each in four configurations: compiled (all tiers), compiled tier-2-only, Approach B, Approach B plus tier 0. Context 8,192, completion cap 512, and p-fin's 5,000 ms per-message latency budget enforced as the policy specifies. Sixteen arms.

Capability ceiling (non-shippable). The same two judge families sent over OpenRouter to six open-weight models (Table 1) with the provider pinned and fallbacks disabled, response_format: json_schema, strict carrying the unchanged schemas, reasoning: {enabled: false}, temperature 0, completion cap 600, and a 60 s per-attempt timeout with retries. Every row records the provider that actually answered beside the one requested. Three full passes over ten thinking-off arms — five models × two families — since GLM refuses to disable reasoning (HTTP 400 on all eleven of its providers) and ran thinking-ON instead.

Table 1. The hosted slate. Providers pinned with allow_fallbacks: false; quantisation and price are the provider's listing (\$ per million tokens, in / out); ran thinking-ON only (\$3.2).

model	provider	quant.	\$ in	\$ out	hosting
deepseek/deepseek-v4-flash-0731	DeepInfra	fp8	0.06	0.18	US
qwen/qwen3.8-flash	Alibaba	—	0.15	0.47	US-route
z-ai/glm-5.3-flash	BaseTen	fp8	0.15	0.50	US
mistralai/mistral-small-2603	Mistral	—	0.15	0.60	EU, first-
nvidia/nemotron-3-super-120b-a12b	DigitalOcean	—	0.30	0.65	US
qwen/qwen3.8-27b	Parasail	fp8	0.24	2.20	US

Thinking ON. GLM-5.3 Flash: the compiled judge at a 600-token cap (one pass; the B family failed that probe 0 of 3), then both families at 8,192 with a 180 s timeout (one pass). The remaining five models were not run thinking-ON: the probe showed their B-family calls taking 46–54 s and aborting, putting a full slate near ten hours; the constraint was time, not budget.

3.3 Metrics, floors and scoring rules

Entity spans are scored span-wise, one-to-one, under exact, overlap and IoU≥0.5 rules. The predicate is scored two ways, and the distinction is this paper's central methodological point: **span-wise**, pairing a finding's span against the gold span; and **message-level**, the boolean the annotators were actually asked — an arm predicts *satisfies* if it emits any predicate finding on that message. Every arm figure is printed beside a floor that uses no model: *first-capitalised-multiword* (the first run of two or more capitalised words) scores 0.571 span-wise and 0.776 message-level; an *orthographic oracle* for entity spans scores 0.454 unbudgeted and 0.695 when told each item's gold count and allowed only its first N hits — a reference that holds label information, and is named as an oracle rather than a floor. **Attempted-only** scoring re-scores an arm and *every floor* over the items that arm was actually allowed to answer, reported beside the whole-gold figures, never instead of them.

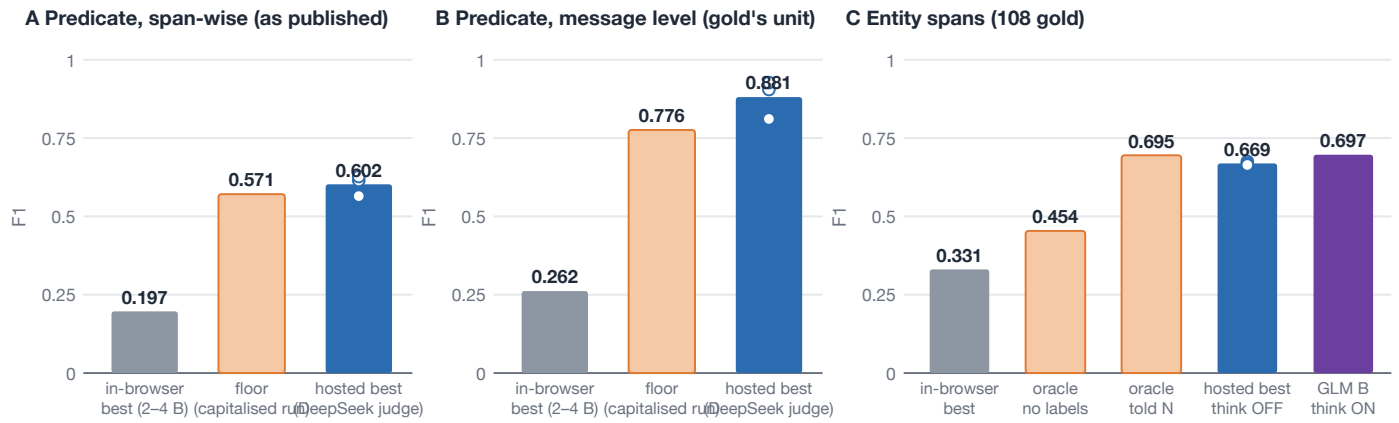
3.4 Provenance and verification

Every record carries the git revision and dirty flag at launch and the IR and policy hashes; records written after the first paid run also carry the completion cap they ran under and the local arms' cap beside it. Spend is reconciled across seven ledger segments against the key endpoint. The driver, scorer and ledger were mutation-tested in three campaigns (23, 20 and 27 of 32 mutants killed, with controls run first and last); the apps/eval suite grew from 791 to 908 tests in the course of this study, and §5 reports what that found.

4 Results

4.1 In-browser arms: the right region, the wrong label

At the predicate, span-wise, the best of sixteen in-browser arms scores **0.197** (compiled Qwen3-4B: 6 tp, 36 fp, 13 fn); ten of the sixteen score 0.000 or are undefined, six of them because Approach B never reaches a first token inside the 5,000 ms budget on three of the four models (0 of 189 answered calls). At message level the best — the same arm — is **0.262**. The mechanism is precision (Figure 6): no in-browser arm exceeds 0.25, and Phi-4-mini reaches recall 0.895 by firing on 125 of 179 messages, 108 of them false positives. On entity spans the best *model-only* arm scores **0.331** (B on Qwen3.5-2B, P 0.581 R 0.231); the eight arms that run tier 0 cluster at 0.444–0.486 whatever model they name — three of them with an LLM that answered zero calls score 0.468 — so tier 0 carries the compiled pipeline's span accuracy and the local model adds recall at the cost of precision. On the smoke corpus the compiled prompt costs 556–1,244 ms p95 to first token against 2,736–2,866 ms for B, on a 5× prompt-size gap (1,410 tokens against 252–280). The single most informative row is a 2 B model placing a finding at exactly [43,59] "Tamarind Grocers", byte-identical to gold, and labelling it an Indian PAN number at confidence 1.



Bars: F1 over all 179 scored items (panels A, B) or all 189 items (C). Hollow dots: the three passes at temperature 0; bar = mean. Orange = a reader with no model.

Figure 2. The headline, three ways. Best in-browser arm, the model-free floor, and the best hosted arm, at (A) the predicate scored span-wise, (B) the predicate scored on the message-level judgment the gold records, and (C) entity spans. Only the metric changes between A and B — same rows, same gold, same arms — and the answer to "is the browser the bottleneck" changes with it. In C, the label-informed oracle (told N) is the reference no thinking-off arm reaches, and the thinking-ON arm is the first to reach it.

Table 2. Every hosted arm, every pass. F1 at the predicate scored span-wise (floor 0.571) and at message level (floor 0.776), and at entity spans (label-informed oracle 0.695, unbudgeted 0.454). Bold blue exceeds the floor or the oracle. J = compiled judge, B = policy-in-context. Unanswered = rows the provider never served (§4.4).

arm	predicate, span-wise (floor 0.571)			predicate, message level (floor 0.776)			entity spans (oracle 0.695 / 0.454)			unanswered
	p1	p2	p3	p1	p2	p3	p1	p2	p3	
J-deepseek	0.565	0.615	0.627	0.811	0.905	0.927	—	—	—	4/1/1
J-qwen-27b	0.561	0.571	0.571	0.760	0.776	0.776	—	—	—	0/0/0
J-qwen-flash	0.340	0.348	0.340	0.450	0.474	0.481	—	—	—	0/0/0
J-nemotron	0.400	0.357	0.222	0.467	0.357	0.296	—	—	—	0/25/20
J-mistral	0.000	0.320	0.091	0.100	0.348	0.190	—	—	—	0/0/0
B-nemotron	—	—	—	0.000	0.000	0.000	0.675	0.669	0.664	0/2/0
B-deepseek	0.000	0.000	0.000	0.261	0.273	0.273	0.570	0.595	0.583	19/4/4
B-qwen-27b	0.000	0.000	0.000	0.273	0.273	0.273	0.585	0.577	0.589	0/0/0
B-mistral	0.227	0.205	0.095	0.400	0.457	0.300	0.526	0.527	0.545	0/0/0
B-qwen-flash	0.174	0.000	0.148	0.261	0.100	0.444	0.520	0.542	0.531	13/0/0

4.2 The ceiling, thinking OFF: level with a regex span-wise, clearly above it message-wise

Table 2 gives every hosted arm in every pass. Span-wise, the compiled judge on DeepSeek scores 0.565, 0.615 and 0.627 against the 0.571 floor — two of three passes clear it, mean 0.602 — and the scorer's own verdict line names it, alone, as beating the floor. Message-wise the same arm scores **0.811, 0.905 and 0.927** against 0.776: all three passes clear it, the last two with perfect recall and three or four false positives across 179 messages. The compiled judge on Qwen3.8 27B sits on both floors in passes 2 and 3 and just below them in pass 1 (0.561/0.571/0.571; 0.760/0.776/0.776), the most stable judge arm (spread 0.016). The improvement over the in-browser arms is large under either reading — 3.1× span-wise at the predicate on three-pass means (0.602 vs 0.197), 2.0× at entity spans (0.669 vs 0.331) — and where it lands relative to the floor is what the metric decides.

The answer to §4.2b depends on a choice the original design did not label as one. The predicate gold is 179 message-level booleans with 19 supplementary spans. Scored span-wise, scale takes these models up to a regular expression and not clearly past it. Scored on the judgment the annotators gave, the ordering is **in-browser 0.262 < floor 0.776 < hosted 0.881** on identical rows, and the gap is substantially the price of the in-browser constraint. Both are real; a stricter metric is not automatically the honest one, and the two answer §4.2b oppositely.

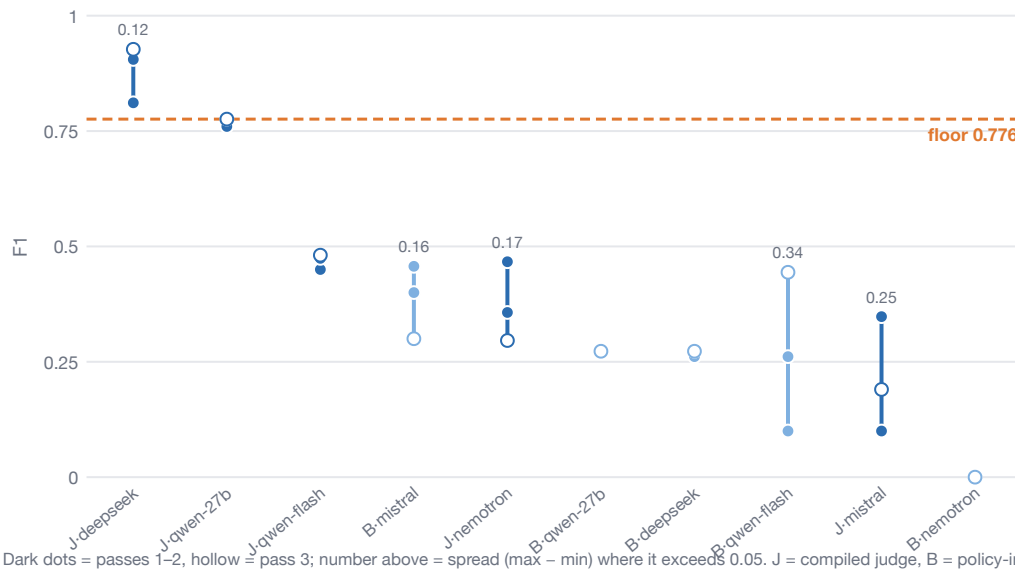
On entity spans only Approach B emits findings. Its best hosted arm, Nemotron, scores 0.675/0.669/0.664 — ahead of the unbudgeted

oracle (0.454) and 0.020 short of the oracle that is told N; every hosted B arm beats the best in-browser model-only arm (0.331), by 1.6–2.0×. **The two families are complementary rather than ranked.** On the message-level entity decision (does this message contain anything sensitive; 108 of 189 do, so always-fire scores 0.727) B's best arm reaches 0.855 while the judge emits no entity spans by construction; on the predicate the judge reaches 0.927 while B collapses to 0.457 or worse, naming the predicate on at most 25 of 189 messages — and never on Nemotron — because it is asked about nine classes at once. That is an argument for the tiered design running both.

4.3 Variance: six times larger where the models are inaccurate

Every call was made at temperature 0 with the provider pinned, so whatever moves between passes is provider and routing variance, not the sampler. It moves a great deal at the predicate and hardly at all at spans (Figure 3): message-level F1 spreads by 0.109 on average across the ten arms and by 0.344 at most — B on Qwen3.8 Flash scored 0.261, 0.100 and 0.444 on identical inputs — while entity-span F1 spreads by 0.018 on average and 0.025 at most. Two consequences follow. A single pass against a hosted provider is not a measurement of classification quality, which is precisely how this study's first headline went wrong. And the instability is *where the inaccuracy is*: these models do not agree with themselves about classification while agreeing about spans to within 0.02, which is *the binding constraint is classification, not span extraction* reached from an independent direction.

A Message-level predicate F1, every hosted arm, three passes at temperature 0



B Pass-to-pass spread

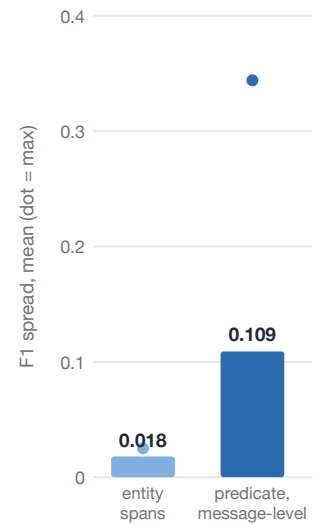


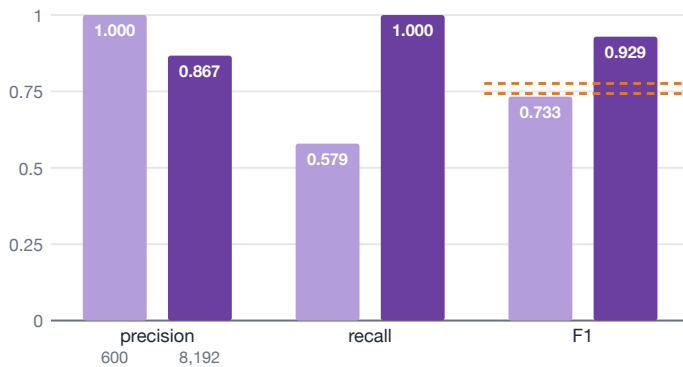
Figure 3. Three passes at temperature 0. (A) Message-level predicate F1 per hosted arm and pass; the compiled judge on DeepSeek rises monotonically and clears the floor every time, the compiled judge on Qwen3.8 27B sits on it, and B on Qwen3.8 Flash swings by 0.344. (B) Mean spread over the ten arms at the two levels; the dot is the maximum.

4.4 Rows the provider never served

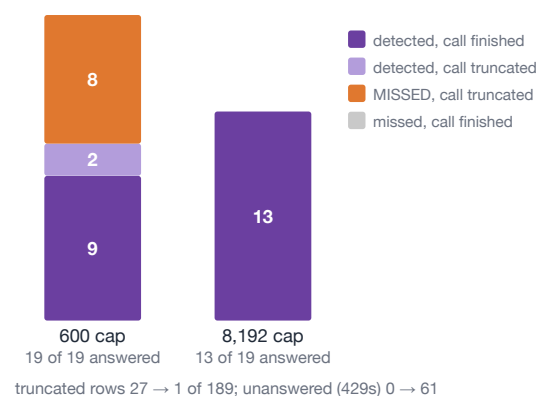
Sixty-eight of 3,780 rows in passes 1–2 (1.8%) carry no call at all: the request exhausted its retries against a 429 rate limit after up to 16.6 s of backoff. Five of them are gold positives, and they are distributed unevenly — thirteen arms lost none, one lost 25 rows. The scorer computes the positive count once over the whole gold, so an item the provider refused to serve was scored identically to one the model read and missed. Re-scoring each arm *and every floor* over the rows that arm answered changes one thinking-off verdict: pass 1 of

the DeepSeek judge moves from 0.565 against a 0.571 floor to **0.578 against 0.565**, so all three passes clear the span-wise floor once arms stop being charged for calls that never happened. The in-browser arms are affected far more — the compiled Qwen3-4B has 70 unanswered rows (10 positives) from budget exhaustion rather than rate limiting — which bears on every in-browser-versus-hosted ratio in this paper. The same rows inflate the item-level p95 latency by up to 2.5× (17,631 ms published against 7,000 ms for work actually done); per-call columns are unaffected.

A GLM judge, thinking ON: 600 vs 8,192-token cap (message level, answered rows)



B The gold positives, by call outcome



Dashed: floors recomputed over the same answered rows (0.776 at 600, 0.743 at 8,192). Every miss at 600 was a truncated call, with no clean misses; at 8,192 the missed list is empty.

Figure 4. Lifting the token cap. (A) GLM-5.3 Flash's compiled judge with reasoning on, at a 600-token and an 8,192-token completion cap, scored at message level over the rows each run answered, with the floor recomputed on the same rows. (B) The gold positives by what happened to the call: at 600 every miss is a truncation; at 8,192 there are none.

4.5 Thinking ON: the cap, not the model

GLM-5.3 Flash cannot disable reasoning. At the 600-token cap, 22.2% of its calls (48 of 216) ended in `finish_reason: length`, and the arm scored 0.571 span-wise and 0.733 message-wise with precision 1.000 and recall 0.579 — eleven true positives and not one false positive. Splitting its 19 gold positives by call outcome (Figure 4B): nine detected on calls that finished, two detected on truncated calls, **eight missed — every one on a truncated call — and no clean misses**. $P(\text{detect} \mid \text{finished}) = 9/9$; $P(\text{detect} \mid \text{truncated}) = 2/10$. The prediction that removing truncation removes the misses was recorded as falsifiable — truncation might merely correlate with difficulty — and then tested.

At 8,192 tokens, truncated rows fall from 27 to 1 and the missed-positive list empties: over the rows it answered, **P 0.867, R 1.000, F1 0.929** against a floor of 0.743 recomputed on the same rows — from 0.042 below the floor to 0.186 above it, with precision giving up the two false positives an arm produces once it stops being silent. Span-wise, attempted-only, it moves 0.571 → 0.611 and the scorer names it among arms whose floor verdict changes (over the whole gold, with 58 rows unanswered, it reads 0.524). Approach B on the same model, which failed its probe 0 of 3 at 600 tokens, becomes the best span-level arm in the study: **0.697 over all 189 rows against the label-informed oracle's 0.695**, and **0.809 against 0.633 over the 130 rows it answered** — the first arm to beat that oracle under either reading (oracle recomputed on the subset with the real implementation, which reproduces 0.695/0.454 on all rows

before being applied). Four bounds: 31% of rows were lost to the provider's shared-pool rate limiting, which was clean fifteen minutes earlier; it is one pass against measured spreads up to 0.344; the B arm took 31.8 minutes for 189 messages and the GLM pair cost 35%

of the entire thinking-off study; and 8,192 was itself hit — once on the judge arm, on 22 rows (15.2% of calls) on the B arm — so no cap-free measurement exists.

4.6 Latency and cost — what transfers to a local server

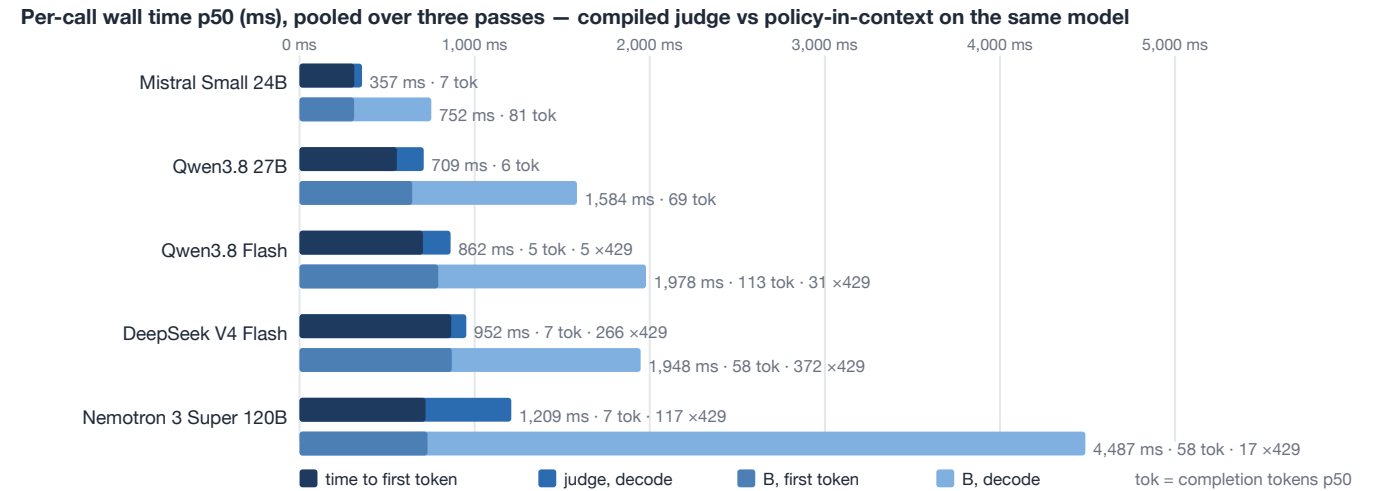


Figure 5. Per-call wall time, judge vs B on the same model, pooled over three passes. The dark segment is time to first token; the remainder is decode. The judge's answer is 5–7 tokens, so its call is almost entirely first-token latency; B's 58–113 tokens make decode the larger share. Labels give completion tokens p50 and the count of 429 rate-limit retries.

Table 3. Latency per hosted arm, pooled over three passes (ms). The decode-rate column is deliberately absent (§5, defect 6). *Item wall is over answered rows only (§4.4). The "s @60 tok/s" column is the transferable number: completion tokens at a local-server decode budget.

arm	calls	TTFT p50	TTFT p95	call wall p50	item wall p50*	compl. tok p50	s @60 tok/s	429s
J-mistral	567	312	563	357	358	7	0.12	0
J-qwen-27b	567	556	871	709	710	6	0.10	0
B-mistral	568	311	592	752	752	81	1.35	0
J-qwen-flash	567	704	4,736	862	954	5	0.08	5
J-deepseek	561	866	3,058	952	1,250	7	0.12	266
J-nemotron	522	719	2,314	1,209	1,221	7	0.12	117
B-qwen-27b	567	643	984	1,584	1,584	69	1.15	0
B-deepseek	541	869	2,975	1,948	2,966	58	0.97	372
B-qwen-flash	555	792	5,796	1,978	2,123	113	1.88	31
B-nemotron	565	731	2,392	4,487	4,551	58	0.97	17

Time-to-first-token and decode rate are provider facts and do not transfer; token counts do. The compiled judge answers this predicate in 5–7 completion tokens; Approach B in 58–113. That is why the judge is uniformly faster on the same model — 357 vs 752 ms on Mistral, 709 vs 1,584 on Qwen3.8 27B, 1,209 vs 4,487 on Nemotron (Figure 5, Table 3) — and why on a DGX-Spark-class 60 tok/s decode budget the judge costs 0.08–0.12 s of decode per message and B 0.97–1.88. Rate limiting is concentrated, not ambient: DeepSeek's two arms absorb 638 of 808 total 429s and four arms saw none. The whole study billed \$1.219 over 6,502 calls (Table 4): the thinking-off slate \$0.900, with B costing 2.05× the judge (its prompt) and an 8.8× spread between the cheapest and dearest model for identical work; the GLM thinking-ON pair alone \$0.316. The price-table estimate feeding the spend guard overshoots billed cost by 64% — safe for a guard, useless for budgeting. Local arms are not directly comparable — different harness, WebGPU load, a page's memory budget — but the same asymmetry holds there: compiled 556–1,244 ms p95 to first token against B's 2,736–2,866, and B unable to answer at all on three of four models within the policy's budget.

4.7 Structured output and reasoning control

Across 5,580 thinking-off calls in three passes, provider-side constrained decoding with the unchanged schemas — bounds included — produced zero malformed or schema-invalid bodies: 5,577 parsed, 3 were truncated at the cap, and there is no third failure mode; 104 calls (1.9%) fell back to a non-streaming transport and still parsed. reasoning: {enabled: false} was honoured on every call by every provider: 5,580 of 5,580 report exactly zero reasoning tokens, with zero nulls — the record keeps "not reported" and

"measured zero" apart, so this is a measured zero everywhere. The span ladder placed findings cleanly at this model size: 6 unresolved quotes and 25 unresolved mentions on 3,799 findings, 0.8 per hundred.

Message-level predicate: precision vs recall, every arm

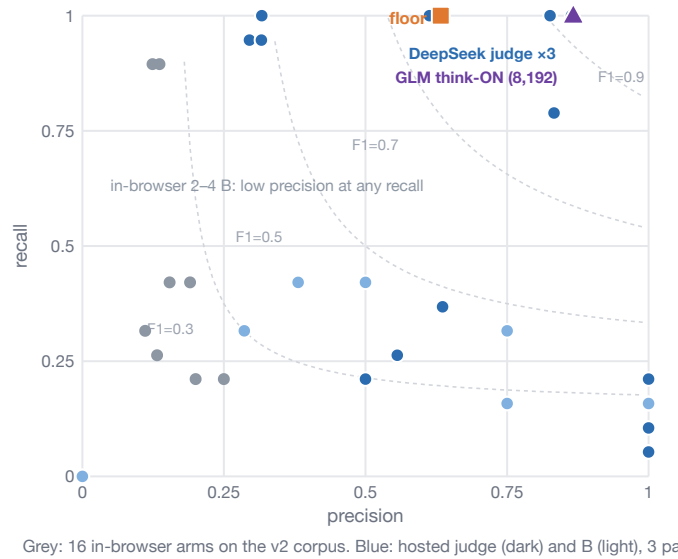
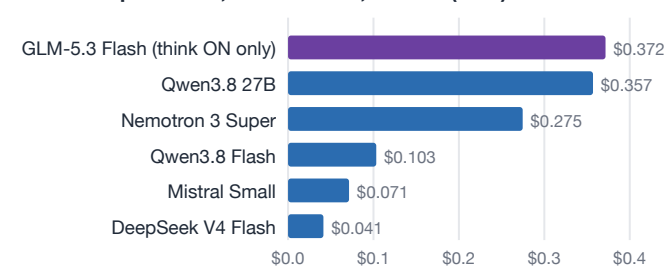


Figure 6. Where the in-browser arms fail. Precision against recall at the message-level predicate for every arm. The in-browser arms (grey) reach any recall only by firing on most messages; the hosted judge (dark blue) separates on precision; the floor (orange) has perfect recall by construction of this corpus.

Billed cost per model, both families, all runs (USD)



Total billed \$1.219 over 6,502 calls, of a \$10 key (12%).

Thinking OFF only: judge \$0.295 vs B \$0.605 — B costs 2.05× (prompt size).
Price-table estimate \$1.475 vs \$0.900 billed: 64% over.

Figure 7. Billed cost. Per model across all runs; the family ratio and estimate error are for the thinking-off slate, the only condition every model ran under.

Table 4. Spend, reconciled. Seven ledger segments joined and checked against the key endpoint; final key reading \$1.29213 of \$10, residual +\$0.073, of which \$0.028 is a probe ledger the authors overwrote by reusing a run id (\$5).

ledger segment	calls	billed
probe	11	\$0.00121
ceiling-01 window 1	1,457	\$0.13645
glmon-01	225	\$0.05572
ceiling-01 window 2	768	\$0.19269
ceiling-02+03	3,756	\$0.51381
thinkon-01 (aborted)	9	\$0.00350
thinkonglm-01	276	\$0.31599
total (seven reconciled segments)	6,502	\$1.21937

5 Defects found in our own instrumentation

These were found by re-deriving published numbers from artifacts and by adversarial mutation review, and are recorded rather than fixed silently because each changes how a number reads.

- **Completion caps did not match.** A docblock asserted the hosted arms' 600-token cap equalled the browser arms'; the browser cap is 512. The 88-token asymmetry favours the ceiling; over 5,580 calls, 12 completions reached 512 and 3 truncated at 600 (0.054%), all in Approach-B arms.
- **The thinking-ON arm could not be measured at the cap.** 22.2% of its answers truncated, and every one of its misses was a truncation (§4.5). Its 0.571 must never be quoted bare.
- **Two wall clocks.** Per-call wall excludes backoff; item wall includes it and includes rows with no call, which inflates its p95 by up to 2.5×. The per-call columns are the ones to quote.
- **Unserved items charged as misses.** 68 rows with no call in passes 1–2 (93 over three), 5 of them positives; attempted-only scoring changes a floor verdict (§4.4).
- **A message-level label scored span-wise.** The gold's 179 booleans were scored through their 19 supplementary spans, presented as "the predicate level". Reporting the level the annotators answered inverts the answer to \$4.2b (§4.2). Now a standing convention: score at the granularity the gold records, and treat disagreement between two legitimate metrics as a finding.
- **Decode rate over n tokens instead of n–1.** Negligible for B (0.8–4.7%), 14–22% for the judge's 5–7-token answers — and the judge's decode window is 9.6–491 ms, so its published 726.7 tok/s measured frame-arrival jitter. The formula is fixed; the column is suppressed below a 100 ms window and flagged above 5% bias.

Process defects were of the same kind. A spend ledger was destroyed twice by reusing a run id — the second time by the authors, on the same day they documented the first — and the fix took three attempts, because a test of the function that returns a filename cannot catch the defect that nobody uses its return value. A docblock justi-

fying the slate's cost ordering was arithmetically false; the order was right on the real workload by 0.20%. And the first headline was drawn from one pass, one metric and one floor, each the least favourable available and none labelled a choice.

6 Discussion

Is the browser the bottleneck? With the conditions stated: against label-informed oracles and span-wise predicate matching, hosted models run by the identical method — 27–120 B where the size is published — land level with a regular expression — the original finding, which survives as a statement about those comparisons. Against the message-level judgment the gold records and against floors with no label access, the best hosted arm separates clearly and repeatably (+0.105 mean over three passes at the predicate, +0.221 at spans) while the in-browser arms stay far below the same floor. On that reading the gap is substantially the price of the in-browser constraint. And with reasoning given room to finish, one model clears the oracle that no thinking-off arm reached, so the thinking-off ceiling is a ceiling on that inference mode rather than on these models.

What does not change under any reading is that the binding constraint is classification, not span extraction. Three independent lines agree: the span ladder resolves 99.2% of findings cleanly at 27–120 B and a 2 B model already places spans byte-exactly; accuracy at spans is competitive with an oracle while accuracy at the predicate is not; and variance across identical passes is 6× larger at the predicate. The in-browser arms' failure is precision — they find the region and fire — and the hosted arms' gain is precision.

For deployment the hosted ceiling is not a configuration: the cloud boundary forbids it. The intended target is a local server (a DGX-Spark-class box running the same open weights), where the transferable numbers are token counts: a compiled judge answers in 5–7 tokens, roughly a tenth of a second of decode per message, against 58–113 for policy-in-context — a 2.05× cost gap that is prompt size and does not shrink with hardware. Mandatory reasoning buys accuracy at a price this corpus makes visible (31.8 minutes for 189 messages on the B family), and a rate-limited shared pool is not a platform to benchmark on.

Threats to validity. One policy, one predicate, one corpus with a known generator leak that gives the capitalisation floor perfect recall; 19 positives, so one item moves recall by 0.053 and the headline arm's F1 by ≈0.03; the in-browser and hosted arms ran in different harnesses; the label-informed oracle is a reference with information no system has; the thinking-ON result is one model, one pass, with 31% of rows missing; hosted quantisation is the provider's, not ours; and the study measured OpenRouter endpoints, not the Spark deployment it argues toward.

7 Conclusion

Run by the identical method, models six to sixty times larger than the shippable ones move policy-conditioned leak detection from far below a trivial floor to clearly above it on the judgment the gold records, tie it on the stricter span-matched reading, and — with reasoning allowed to finish — beat a label-informed oracle on spans. The browser constraint is therefore a real part of what costs accuracy, the binding constraint at every scale is classification rather than span extraction, and the compiled-judge design is the right one to carry to a local server: it is cheaper, faster and better at the semantic predicate than policy-in-context, while policy-in-context is the better entity finder, so a deployment runs both. Every number here regenerates from artifacts; the study's most useful product may be the six ways its own instrumentation was found wrong.

Reproducibility. Repository `orkancelikhisar/ai-dlpp` (private), branch `feat/tier2-judge`: `driver pnpm -C apps/eval ceiling, scorer ceiling:score, ledger ceiling:ledger`; this paper's data and figures regenerate with `docs/paper/extract-numbers.py` and `make-figures.py` from `runs/` and `corpora/generated/`. Full records in `docs/research/`. The API key was revoked after the study; its final reading is recorded, not reproducible.